ACI-CO Repository Architecture

Colombian Actuarial Climate Index (ACI-CO) pipeline

August 10, 2026

Abstract

This document is a LaTeX companion to `ARCHITECTURE.md` and `CLAUDE.md` at the repository root. It describes how the ACI-CO codebase is organized: the scientific target it computes, the actual data flow between scripts (as opposed to the README's idealized version), where the repository's documentation is authoritative versus stale, and how to read already-processed outputs from a notebook (e.g. Google Colab) using the `aci_lib` data-access library introduced alongside this document. It is a description of the codebase as it stands, not a specification – where the code and the docs disagree, the code wins.

Contents

1 What this repository is

This is a research pipeline that builds the **Colombian Actuarial Climate Index (ACI-CO)** from ERA5(-Land) reanalysis data (1961–2024): a Colombian adaptation of the Actuaries Climate Index (ACI) framework, extended with ENSO calibration, a bootstrap uncertainty band, multi-dataset robustness checks, and validation against Colombia's national disaster registry (UNGRD).

It is an active research codebase, not a packaged library: there is no test suite, several parallel/experimental implementations of the same pipeline stage coexist deliberately, and a non-trivial share of the *working* pipeline was never committed to git (Section 7).

Authoritative methodology source. The methodology (how T90, T10, Rx5day, CDD and WP are computed; how the composite, ENSO decomposition, and bootstrap bands work) is described in `articles/aci_co_submission/article1.tex`, §3–§7 – *not* the older root-level `articles/article1.tex` (a superseded LOWESS-based draft), and not the empty stubs in `docs/methodology.md` / `docs/results.md` / `docs/zones_description.md`.

2 The scientific target: ACI-CO

The ACI-CO composite is the unweighted mean of five signed, standardized components (divided by $K = 5$; an earlier draft divided by 6 for a placeholder sea-level component, since dropped for lack of tide-gauge coverage):

Component	Meaning	Sign	Threshold
T90	Fraction of days/month with temperature above the 1961–1990 baseline 90th percentile	+	Empirical P90 per grid cell × calendar month
T10	Fraction of days/month with temperature below the baseline 10th percentile	−	Empirical P10
Rx5day	Max 5-day precipitation total per month, z-scored vs. 1961–1990 climatology	+	z-score vs. monthly climatology
CDD	Annual max consecutive dry days (<1 mm), interpolated onto the monthly axis	+	—
WP	Wind-power ($\propto U^3$) exceedance	+	Empirical P90 (current); replaces an earlier parametric $\mu + 1.28\sigma$ threshold, discarded because WP is right-skewed

2.1 Extensions beyond the base ACI framework

- **ENSO-neutral decomposition** (§4): asymmetric ARIMAX regression per component/region on Niño-3.4 ($E^+ = \max(E, 0)$, $E^- = \min(E, 0)$) plus a linear trend and AR(p) errors, separating secular warming from ENSO-driven variability.

- **Bootstrap uncertainty bands** (§4.4): block-resample ($L = 12\,\text{mo}$) the 1961–1990 baseline composite into $B = 500$ (national) / $B = 200$ (regional) null-model replicates, testing whether the observed trend exceeds baseline-period variability. Replaced an earlier LOWESS residual-envelope approach.
- **Multi-dataset robustness** (§5): ERA5-Land vs. ERA5 standard vs. CHIRPS.
- **UNGRD validation** (§7): negative-binomial regression of disaster counts (floods, landslides, windthrow, vegetation fires – 49,734 events, 1998–2022) on ACI-CO components.

2.2 Regions actually computed

Despite the paper conceptually discussing 5 natural regions and 32 departments, the implementation aggregates **4 units**: national (area-weighted, all of Colombia) and 3 departments chosen for insurance relevance – **Antioquia**, **Cundinamarca-Bogotá**, and **Valle del Cauca**. Shapefiles also exist for Bogotá, Medellín, Cali, San Andrés/Providencia, Pacífico, and Amazonas, but the corresponding `data/processed/anomalias_{bogota,medellin,san_andres_providencia}/` directories are empty – never populated.

3 Repository layout

Path	Contents
`src/scripts/`	The original OPT pipeline: download, percentile, and anomaly-calculation scripts (`calcular_*`, `ecmwf_descarga.py`, `unir_archivos.py`, `graficas.py`, ...). Spanish-language docstrings dominate here.
`src/forecast_-scripts/`	Two independent forecasting families (AutoTS ensembles vs. SARIMAX/ETS + ENSO) plus `common/`, the shared modules extracted from both (Section 4.2). English-language, newer code.
`src/dashboard/`	`app.py` – a read-only Streamlit dashboard that reads Stage 3/5 CSVs directly; not a pipeline stage.
`src/utils/`	Shared helpers (`get_available_years`, `get_cached_shapefile`), consolidated from near-identical copies previously duplicated across the `calcular_*` scripts.
`src/aci_lib/`	Read-only data-access library for notebooks (Section 8) – added alongside this document, not part of the original OPT/forecast pipeline.
`data/raw/`, `data/processed/`	Pipeline inputs/outputs, entirely gitignored except for `.gitkeep` placeholders (Section 7).
`articles/`	Three papers at different stages: the current submission (`aci_co_submission/`), a superseded LOWESS draft (`article1.tex`), and a separate forecasting report (`AnnualICAForecast_Report.tex`).
`docs/`	Mostly empty stubs (`methodology.md`, `results.md`, `zones_description.md`); the one substantive file is `Diccionario.md`, a ∼2400-line data-source vetting document.

Path	Contents
tests/	Placeholder only (test.txt); there is no test suite or CI.

4 Actual data flow

The README describes an idealized 8-step flow; several of the scripts it names do not exist (Section 6). The flow below is reconstructed from what the scripts actually read and write.

Stage	Script(s)	Output
0. Download	ecmwf_descarga.py (canonical), sst_descarga.py, get_aux_data.py, IDEAM station data via pyreadr	data/raw/era5/*.grib, SST GRIB, data/raw/auxiliary/, data/raw/ideam/*.rds
1. Merge / resample	unir_archivos.py; independently, the untracked run_aci_colombia.py re-merges raw GRIB into a from-scratch reimplementation	era5_daily_combined_{tmp,rain,wind}.nc; data/processed/aci_daily/, aci_colombia_1961_2024.csv
2. Percentiles (1961–1990 baseline)	calcular_percentil_{temperatura,lluvia,viento}.py	era5_*_percentil.nc, percentiles*.csv
3. Anomalies, per region	calcular_anomalias_{temperatura,lluvia,viento}.py, orchestrated by calcular_anomalias_regiones.py over data/shapefiles/*.shp	data/processed/anomalias_<region>/anomalies_{temperature,precipitation,drought,wind}_combined.csv (populated: antioquia, cali, colombia, cundinamarca_bogota, valle_cauca; empty: bogota, medellin, san_andres_providencia)
4. Daily variants	produce_daily_anomalies.py, append_2025_2026.py (untracked)	data/processed/daily_anomalies/*.xlsx
5. ENSO / SST index	anomalias_sst_daily.py, compare_enso_flags.py	sst_index_colombia_pacific.csv, enso_flag_comparison.csv, validated against NOAA CPC oni.ascii.txt
6. Downscaling	downscale_era5_temperature.py, downscale_era5_tp.py (RandomForest, DEM/NDVI/IDEAM)	data/processed/downscaled/, validated in data/processed/era5_ideam_comparison/
7. Visualization / export	graficas.py, generate_correlation_matrices.py	articles/graficas/<region>/*.png, xlsx exports, correlation heatmaps
8. Forecasting	Two independent families – see Section 4.2	articles/graficas/forecast_*/

data/indices/ and data/zones/ exist but are effectively empty – no script currently writes to those exact paths.

Raw ERA5 .grib backup. data/raw/era5/*.grib is gitignored (Section 7) and not reproduced anywhere in version control. A backup copy of these raw downloads is kept on Google Drive: https://drive.google.com/drive/folders/17SdbsxVJYnqjPzIlS4FFZ6MowfMcHfqW.

4.1 Two parallel "correctness check" implementations

There are effectively two independent implementations of the core ACI algorithm in this repository:

- **The OPT pipeline** (src/scripts/calcular_*, tracked in git, the README's documented flow) – the original project pipeline, per-region, percentile/anomaly based.
- run_aci_colombia.py (untracked) – a from-scratch reimplementation of the ACI-Python reference algorithm directly on raw GRIB, used to cross-validate the OPT pipeline's output. compare_aci_colombia.py and compare_repos.py (also untracked) diff the two and document the algorithmic differences.

Neither is "the old version" of the other – they are independent implementations kept deliberately in sync for cross-validation, and this comparison grounds the paper's "validation against x-ACI" discussion (§8).

4.2 Forecasting: two families plus shared modules

Two independent forecasting families exist side by side and are *not* a superset/subset of each other:

- forecast_ica_{monthly,daily}.py – AutoTS ensembles.
- The ETS(X)_* family – statsmodels SARIMAX/ETS with an ENSO exogenous regressor.

src/forecast_scripts/common/ holds logic that was byte-identical or near-identical across these scripts before consolidation:

Module	**Extracted from**	**Used by**
data_loading.py	Byte-identical in forecast_ica_monthly.py and ETS_ica_forecast.py	Both, via thin same-named wrapper methods
climate_index.py	Near-identical in all three monthly forecasters	All three
stationarity.py	Near-identical in ETS_ica_forecast.py / ETSX_ica_forecast.py	Both
diagnostics.py (ModelErrorDiagnostics)	Moved unchanged from the old error_diagnostics.py	Not currently called by anything

Deliberately *not* merged (structurally parallel, not literal duplicates): the ONIDataHandler /ONIDataHandlerDaily ENSO-fetching classes, the SARIMAX+Lasso fitting methods, the AutoTS setup (same shape, different hyperparameters), and all visualize_forecasts-style plotting methods (4 separate implementations, each with different overlays).

5 Documentation status

Document	Status
docs/methodology.md, docs/results.md, docs/zones_-description.md	Empty stubs. Methodology lives in articles/aci_co_submission/article1.tex instead.
docs/explicacion_-excel.md	One truncated sentence, no body.
docs/data_dictionary.md	Early, incomplete fragment (HUMBOLDT-only) of docs/Diccionario.md.
docs/Diccionario.md	The real, complete data-source vetting document (∼2400 lines, 20 candidate sources evaluated). Canonical over data_dictionary.md.
README.md	Describes an idealized 8-step flow; two referenced scripts do not exist.

6 Discrepancies between README/docstrings and reality

- README step 1 says to run `descargar_datos.py` – it does not exist; `ecmwf_descarga.py` is the actual entry point.
- README step 6 says to run `sealevel.py` – does not exist. `sealevel2.py` exists but does not do sea-level analysis; it is a Mann-Kendall trend + Q-Q normality check on precipitation-like series with leftover Jupyter cell markers, suggesting it was pasted from a notebook and misnamed. Sea level is not part of the current ACI-CO methodology.
- `src/scripts/earthub_descarga.py` has a whole-file indentation error and cannot run as-is, on top of embedding a live-looking credential (Section 7.1).
- `src/scripts/display.py` is not a plotting module – it is a Greece-precipitation download script (also embeds the same credential) that produced a 562 MB `.nc` file at the repo root as a side effect.
- Stale internal docstrings/comments: `era5datasets.py` says `"""compare_land_only.py"""`; `get_aux_data.py`'s header says `get_auxiliary_data.py`. Trust the filename/git path.
- `graficas_altaresolucion.py` is a 4-line dead stub.
- `ETSX_seasonal_motif_forecast.py` (the "broken two-stage Lasso→ETS" approach `ETSX_daily_ica_forecast.py`'s docstring says it replaced) has been deleted – confirmed unused before removal.
- `example_grid_search_era5.py` imports `ONIDataHandler` from `ETSX_ica_forecast.py` expecting methods that do not exist on that class – a pre-existing break, unrelated to the forecast_scripts modularization.
- `articles/AnnualICAForecast_Report.tex` references `scripts/daily_forecast_to_annual.py` and `scripts/ar1_vs_ets_final.py` – neither exists anywhere in the repository. Treat that report's pipeline as documented-but-unimplemented.

7 The git tracking gap

A significant portion of the *current, more-correct* pipeline exists only in the working tree and has never been `git added`: `run_aci_colombia.py`, `compare_aci_colombia.py`, `compare_repos.py`, `create_region_shapefiles.py`, `probe_2025.py`, `produce_daily_anomalies.py`, `append_2025_2026.py`, `reproduce.py`, `plot_repo_anomalies.py`, and most of `articles/` (the `aci_co_submission/` folder, `produce_paper_plots.py`, `build_validation_outputs.py`, etc.).

Conversely, things that *are* committed include generated junk (`presentation.aux/log/out`, `texput.log`, `output.log`, `comparison.png`, `storm_daniel.png`, `.vs/`, `proyecto-indices/`, root-level `.nc`/`.csv` data files) and a `.gitignore` that was itself accidentally overwritten with a PowerShell heredoc command instead of its output.

Untracked here often means "uncommitted work in progress," not "disposable." Do not delete untracked files without checking which side of this split they are on.

7.1 Known secrets in the repository

Two Earth Data Hub PAT tokens are hardcoded and **committed** in `src/scripts/display.py` (line 13) and `src/scripts/earthub_descarga.py` (line 15) – the same token, duplicated. A CDS API key also appears in `GRID_SEARCH_AND_ERA5_GUIDE.md` (line 44). These require rotation/history-scrubbing independent of any code change. New download scripts should follow the pattern in `ecmwf_descarga.py` / `sst_descarga.py`: `cdsapi.Client()` reads credentials from the user's local `~/.cdsapirc`, nothing embedded in source.

8 The `aci_lib` data-access library

`src/aci_lib/` is a small, dependency-light Python package that turns `data/processed/` into a browsable, loadable catalog, so that already-computed pipeline outputs can be pulled into a notebook (locally or in Google Colab) without knowing the directory's internal layout by heart.

8.1 Why it exists

`data/processed/` mixes single tabular files, single gridded NetCDF files, and long runs of per-year or per-year-month NetCDF files that really form one logical time series (e.g. 768 files under `anomalias_colombia/anomalies_temperature_1961_1.nc` through `_2024_12.nc`). `aci_lib` groups the latter automatically and exposes a uniform three-function API:

- `list_datasets()` – a DataFrame of every dataset name, kind, file count, and (where known) a description sourced from this document's data-flow tables.
- `describe(name)` – the underlying file list and kind for one dataset.
- `load(name)` – returns a `pandas.DataFrame` for CSV/XLSX datasets, an `xarray.Dataset` for single or multi-file NetCDF datasets (opened lazily via `xarray.open_mfdataset`), or a file path for images/raw text files.

8.2 Important caveat: data is not in git

`data/processed/` is entirely gitignored (Section 7) – only a `.gitkeep` placeholder is tracked. A plain `git clone` of this repository, including in Colab, brings the *code* in `src/aci_lib/` but *not* the data it reads. The data must be supplied separately: mount it from Google Drive, upload and unzip an archive of `data/processed/`, or copy it in via any other means available in the notebook environment.

8.3 Quickstart in Google Colab

```
# 1. Get the code (data/processed/ is gitignored, so this clone
# does NOT include the data -- only src/aci_lib/).
!git clone https://github.com/<org>/aca_indice_climatico_opt-main.git
import sys
sys.path.insert(0, "/content/aca_indice_climatico_opt-main/src")

# 2. Supply the data. Pick ONE of:
# (a) mount Drive, if data/processed/ was uploaded there beforehand:
from google.colab import drive
drive.mount("/content/drive")
DATA_DIR = "/content/drive/MyDrive/aca_data/processed"

# (b) upload + unzip a local copy:
# from google.colab import files
# files.upload() # e.g. processed.zip
# !unzip -q processed.zip -d /content/aci_data
# DATA_DIR = "/content/aci_data/processed"

import aci_lib
aci_lib.set_data_dir(DATA_DIR)

# 3. Browse and load.
catalog = aci_lib.list_datasets()
catalog.head(20)

national_composite = aci_lib.load(
    "anomalias_colombia/anomalies_temperature_combined"
) # -> pandas.DataFrame

gridded_series = aci_lib.load(
    "anomalias_colombia/anomalies_temperature"
) # -> xarray.Dataset, lazily concatenated from ~768 monthly files
```

Run locally, inside a checkout of this repository, `set_data_dir()` is unnecessary: `aci_lib.get_data_dir()` walks up from `src/aci_lib/` to find the repository root (marked by `CLAUDE.md`) and locates `data/processed/` on its own.

8.4 Scope

`aci_lib` currently covers everything under `data/processed/` only – it does not reach into `articles/*/graficas/paper_figures/` (the paper's final figures/tables) or `data/raw/`. Both are reasonable follow-up extensions if a notebook workflow needs them, but were kept out of scope to keep the catalog's grouping logic simple and its descriptions trustworthy.

9 Benchmarking the pipeline in Colab

Unlike `aci_lib` (read-only over already-computed `data/processed/`), this capability runs the actual pipeline *stages* – Stage 1 merge/resample, Stage 2 baseline percentiles, Stage 3 anomalies – against raw ERA5 `.grib`, to see how long they take on whatever compute a given Colab session provides.

9.1 Why no script changes were needed

`unir_archivos.py`, `calcular_percentil_temperatura.py`, and `calcular_anomalias_temperatura.py` already expose plain, parameterized functions (`process_yearly_data_tmp`, `calcular_percentiles`, `procesar_anomalias_temperatura`, ...) behind a thin `__main__` CLI wrapper that just supplies default repo-relative paths. **They were already importable as a library** – calling them directly from a notebook with explicit paths works without modification. This was verified by importing `unir_archivos` and timing `process_yearly_precipitation_data` against a real local `.grib` file before writing the Colab notebook described below.

9.2 `src/drive_sync.py`

A new, small helper module (not part of `aci_lib`) for pulling data out of the shared Google Drive folder (Section 7) into this repo's `data/` layout, for use inside Colab. That folder (`My Drive/2. Datos`, confirmed from the Drive UI – it is a shortcut sitting directly in `My Drive` root; "Indice Climatico Actuarial", seen in Drive's breadcrumb/location panel, is the owner's canonical path, not where the shortcut lives) is shared with specific named collaborators, not "anyone with the link" – confirmed from its "Who has access" panel, which is also why anonymous fetching (tested with `gdown`) returns HTTP 401. So `drive_sync.sync(...)` instead copies (or symlinks) from an already-mounted `google.colab.drive`, which authenticates as whoever is logged into that Colab session.

Its top-level `DEFAULT_MAPPING` (`shapefiles`, `salidas_colombia`, `salidas_cundinamarca_bogota`, `datos_indice` → their `data/` equivalents) is confirmed against the real Drive listing. `era5/` is deliberately left out of the default mapping: it turned out not to be a flat folder of raw `.grib` files as first assumed, but its own tree (`completos/`, `Combined/`, `percentiles_nc/`, `Subsets para excel/`, `union/`, `ejemplo_4_73/`, `otros/`), and which of those holds the flat `era5_{tmp,rain,wind}_<year>.grib` files the pipeline scripts expect is not yet confirmed – see `notebooks/explore_drive_structure.ipynb` in the `aca_aci_collab` repo, a one-off Drive-tree dump built for exactly this.

9.3 `aca_opt_benchmark.ipynb`

Lives in the `aca_aci_collab` companion repo's `notebooks/`, alongside `aci_lib_quickstart.ipynb`, but clones *this* pipeline repo rather than the companion repo, since that's what has the scripts to benchmark. It fetches one year of raw temperature `.grib` (not the whole ∼13 GB archive) and times Stages 1–3 against just that year, then extrapolates linearly to a full 1961–2024 (64-year) run for Stages 1 and 3. Stage 2 (baseline percentiles) is timed against the same single year purely as a computational-shape proxy – the real pipeline computes it once over the fixed 30-year 1961–1990 baseline, not once per year, so its extrapolated figure isn't meaningful; multiply the single-year Stage 2 time by roughly 30 instead.

10 Cleanup priorities (standing list)

Ordered roughly by urgency/impact; see `ARCHITECTURE.md` for the canonical, living version of this list.

1. **Secrets**: rotate the Earth Data Hub PAT tokens and the CDS key (Section 7.1); scrub from git history if the two committed copies need to be fully removed.
2. **Commit the real pipeline**: the untracked files listed in Section 7.
3. **Purge committed generated junk**: build artifacts and data files committed at the repo root instead of under `data/`.

4. **Fix `.gitignore`**: currently wraps the real ignore rules in a literal PowerShell heredoc command that got committed instead of its output.

5. **Delete disk-only bloat** (not committed, safe to remove locally): environment archives, unrelated test data, `__pycache__/`.

6. **Consolidate duplicated helpers** into `src/utils/` (mostly done – see `get_available_years`, `get_cached_shapefile`).

7. **Resolve naming/content mismatches**: `display.py`, `sealevel2.py`, stale internal docstrings.

8. **Decide the fate of superseded scripts**: `graficas_altaresolucion.py`, `docs/data_dictionary.md`, root `articles/article1.tex`.

9. **Fill or drop the empty doc stubs** in `docs/`.

10. **Populate or drop** the empty `data/processed/anomalias_{bogota,medellin,san_andres_providencia}/` directories and the unused `data/indices/`, `data/zones/` locations.

Confirm scope with a maintainer before acting on any of these – several things that look like dead scratch code are load-bearing work that simply never got committed (Section 7).